Ders 12 Çalışma Özeti

Ders 12 Çalışma Özeti

Genel Konular

- Dersin tarihi 1 Kasım 2021 Pazartesi. Ders çevrim içi yapıldı.
  - Asıl anlatım yaklaşık 00:10:30'da başlar. Öncesi bekleme ve bir öğrencinin `fork` sorusudur.
  - Bir önceki haftanın Salı dersi yapılmadı. Bu grup diğer gruplardan biraz önde.
  - Slayt 3.18'den 3.38'e kadar işlendi.
- Zamanlayıcıların tekrarı: kısa, uzun ve orta vadeli zamanlayıcı, konvoy etkisi (convoy effect).
- Mobil sistemlerde çok görevlilik (multitasking): iOS ve Android.
- Bağlam değişimi (context switch) ve maliyeti.
- Süreç işlemleri (operations on processes)
  - Süreç yaratma: `fork` ve `exec`, ebeveyn-çocuk (parent-child) ilişkisi, kaynak paylaşımı seçenekleri.
  - Linux süreç ağacı: `init` (PID 1) ve çocukları.
  - `fork` dönüş değeri ile ebeveyn ve çocuğu ayırma; C örnekleri.
  - `exit` ve `return` farkı.
  - Süreç sonlandırma (process termination): `exit`, `abort`, `kill`, düzgün kapanış (graceful shutdown), `atexit`.
  - Zombi (zombie) ve öksüz (orphan) süreçler, `wait`.
- Çok süreçli mimari örneği: Google Chrome.
- Süreçler arası iletişim (IPC, interprocess communication): bağımsız ve işbirlikçi (cooperating) süreçler, paylaşımlı bellek (shared memory) ve mesaj geçirme (message passing).

Hocanın Özellikle Vurguladığı Kısımlar

- Bağlam değişimi bir ek yüktür (overhead).
  - Bağlam değişimi sırasında CPU kullanıcı uygulaması değil, işletim sistemi kodu çalıştırır. Bu sürenin olabildiğince kısa olması gerekir.
- `fork` sonrası iki süreç birebir aynıdır, ama iki bilgi mutlaka farklıdır.
  - Biri süreç kimliği (PID). Her sürecin kimliği tektir.
  - İkincisini hoca öğrencilere düşünmeleri için sordu. Cevap bir sonraki derste verildi: ebeveyn süreç kimliği (parent PID).
- `fork`'un dönüş değeri ebeveynde çocuğun PID'si, çocukta 0'dır.
  - Bu sayede kaynak kodda ebeveyn ve çocuk ayrılır, ikisine farklı işler yaptırılır.
- `fork` sonrası önce hangi sürecin çalışacağı bilinemez. Buna zamanlayıcı karar verir.
- `wait` belirli bir sırayla dönmez. Bitmiş çocuklardan herhangi biri için döner.
- İşletim sistemi IPC mekanizmalarını sunmaktan sorumludur, doğru kullanımından değil.
  - Senkronizasyonu yanlış yaparsanız açlık (starvation) ya da kilitlenme (deadlock) oluşabilir. İşletim sistemi kilitlenmeyi çözse bile programınızdaki mantık hatası sürer.
- Derste soru sorarken net konuşun ve dersin içeriğinde kalın.
- **Uygulamalar (laboratuvar):**
  - Bir aksilik olmazsa 8 Kasım haftasında başlayacak. İlk uygulama kabuk (shell) ve Linux komutları üzerine.
  - Uygulama saatleri not verilen saatler değil. Uygulama sonunda test ya da sınav yok.
  - Uygulamada anlatılanlar vize ve finalde sorulacak.
- MPI derste ayrıca anlatılmayacak; yalnız genel olarak değinilecek.

Kısa Tekrar Notları

- Kısa vadeli zamanlayıcı: sıradaki CPU döngüsünde çalışacak süreç.
- Uzun vadeli zamanlayıcı: diskten belleğe alınacak işler. G/Ç ağırlıklı ve CPU ağırlıklı süreçler dengelenmezse konvoy etkisi oluşur.
- Orta vadeli zamanlayıcı: swap out / swap in.
- Mobil sistemler güç kısıtlıdır. Eski iOS'ta yalnızca bir ön plan (foreground) uygulaması çalışırdı. Android'de arka plan işi bir servis (service) ile yapılır.
- Bağlam = program sayacı + yazmaç seti + kullanılan kaynaklar. Bağlam PCB'de saklanır.
- Bağlam değişimi süresini işletim sistemi tasarımı ve donanım desteği belirler.
- Süreç yaratma = `fork` (kopya süreç) + `exec` (yeni programı yükleme).
- `fork` dönüşü: `< 0` hata, `== 0` çocuk, `> 0` ebeveyn (değer çocuğun PID'si).
- `getpid()` sürecin kendi PID'sini, `getppid()` ebeveyninin PID'sini verir.
- `exit` süreci her yerden sonlandırır. `return` yalnızca içinde bulunulan fonksiyondan çıkar. `main`'den `return` neredeyse `exit` gibidir.
- Bir iş parçacığından `exit` çağrılırsa bütün süreç biter. Yalnız iş parçacığını bitirmek için `pthread_exit`.
- `atexit(f)`: süreç sonlanırken `f` fonksiyonunun çalışmasını sağlar (kaynak temizliği).
- Zombi: çocuk bitti, ebeveyn `wait` ile sonucunu almadı. `ps` çıktısında `<defunct>`.
- Öksüz: ebeveyn önce bitti. Çocuğun ebeveyni `init` (PID 1) olur, sonucunu `init` toplar.
- IPC iki temel yöntem: paylaşımlı bellek ve mesaj geçirme.
- Mesaj geçirme: dağıtık sistemler ve küçük veriler için uygun; her mesajda sistem çağrısı gerekir.
- Paylaşımlı bellek: büyük veriler için hızlı; yalnız oluştururken sistem çağrısı gerekir, sonra doğrudan okunup yazılır.

Detaylı Açıklamalar

Zamanlayıcıların tekrarı ve konvoy etkisi

Uzun vadeli zamanlayıcı belleğe olabildiğince çok süreç almak ister, ama süreçler arasında denge olmalıdır. Hepsi CPU ağırlıklı ya da hepsi G/Ç ağırlıklı olursa süreçler farklı kuyruklarda arka arkaya dizilir. Buna konvoy etkisi denir.

Orta vadeli zamanlayıcı, bellekte duran ve şu an çalışmayacak süreçleri ikincil belleğe çıkarır, çalıştırılmak istenenleri belleğe alır. Bu çoklu programlamayı (multiprogramming) daha verimli yapar.

Mobil sistemlerde çok görevlilik

Mobil cihazlar sınırlı bir güç kaynağıyla çalışır. Bir telefonun bir şarjla mümkünse iki gün dayanması beklenir. Bu yüzden güç tüketimi işletim sistemi tasarımında bir ölçüttür.

- **iOS (eski sürümler):** süreçler ön plan ve arka plan diye ikiye ayrılır. Ön planda yalnızca bir uygulama etkin olabilirdi. Arka planda yalnızca sistemin sunduğu bazı hizmetler çalışabilirdi. Kullanıcı uygulamalarının arka planda çalışması sonradan açıldı.
- **Android:** böyle bir kısıt yok. Bir uygulamanın arka planda çalışması gerekiyorsa bir servis (service) yazılır. Servis bağımsız bir bileşendir ama uygulamanın temsilcisi gibi çalışır.
- Arka plan işi kısa ve periyodik (ör. yeni e-posta kontrolü) ya da uzun soluklu (ör. arka planda müzik) olabilir.

Bağlam değişimi

Bir sürecin bağlamı (context), yaşamı boyunca dokunduğu ve kullandığı her şeydir:

- program sayacı (sıradaki komut),
- yazmaç seti (komutların üzerinde çalıştığı veriler),
- o an kullandığı kaynaklar.

Bağlam PCB'de saklanır. Çalışan süreci CPU'dan alıp başkasını koymak için:

1. Çalışan sürecin bağlamı PCB'sine kaydedilir.
2. Seçilen sürecin bağlamı PCB'sinden CPU'ya yüklenir.

Bağlam değişimi sırasında kullanıcı uygulaması çalışmaz; CPU işletim sistemi kodunu çalıştırır. Yani bu süre bir ek yüktür. Normalde 1-2 birim sürmesi gereken iş 20-30 birim sürerse CPU boşa harcanır.

Süreyi etkileyen etkenler:

- **İşletim sistemi tasarımı:** PCB'de ne kadar bilgi tutulduğu, nereye ve nasıl kaydedildiği.
- **Donanım desteği:** Bazı işlemcilerde birden fazla yazmaç seti vardır. Hoca SPARC işlemcileri örnek verdi: birden fazla yazmaç kopyası olduğu için bağlam değişimi tek bir komutla yapılabiliyordu. Ama süreç sayısı yazmaç seti sayısını geçince bağlam değişimi yine donanım desteği olmadan yapılmak zorunda kalır.

Süreç yaratma: fork ve exec

İşletim sistemi açılırken `init` süreci diskten belleğe kopyalanır ve başlatılır (PID 1). Sistem hizmetleri şöyle başlatılır:

1. `fork` ile kendisinin bir kopyası yaratılır.
2. `exec` ile istenen programın kodu bu yeni sürece yüklenir.

Linux süreç ağacı örneğinde `init`, `kthreadd`, `sshd` ve `login` gibi süreçleri yaratır. `login` bir kabuk (`bash`) başlatır. Kabuktan `ps` ve `emacs` çalıştırılmıştır. Ağaçtaki her çizgi bir `fork` + `exec` demektir.

Kaynak paylaşımı seçenekleri:

- Ebeveyn ve çocuk bütün kaynakları paylaşır (`fork`'un hemen ardından durum budur).
- Çocuk, ebeveynin kaynaklarının bir kısmını paylaşır.
- Hiç kaynak paylaşmazlar.

`fork`'tan sonra kaynakların nasıl ayrışacağını yazdığınız kod ya da sürece yüklediğiniz program belirler.

Çalışma biçimleri:

- Ebeveyn çocuğu yaratır ve çocuk bitene kadar bekler (`wait`). Kabuk böyle çalışır: komut okur, çocuk süreç yaratır, programı `exec` ile yükler, sonucu gösterir, yeni komut bekler.
- Ebeveyn ve çocuk birbirinden bağımsız olarak çalışmaya devam eder. Hoca bunu "saldım çayıra, Mevlam kayıra" diye özetledi.

fork dönüş değeri

`fork` çağrısı bittiği anda iki süreç vardır. İkisi de `fork`'tan sonraki satırdan devam eder.

Dönüş değeri	Anlamı
`< 0`	Hata; yeni süreç yaratılamadı
`0`	Çocuk süreçtesiniz
`> 0`	Ebeveyn süreçtesiniz; değer çocuğun PID'si

Çocuk için dönüş değeri 0'dır, çünkü çağrıyı o yapmadı; çağrıdan sonra oluştu.

Slayttaki C programı:

```c
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>

int main()
{
    pid_t pid;

    /* fork a child process */
    pid = fork();
```

```c
    if (pid < 0) { /* error occurred */
        fprintf(stderr, "Fork Failed");
        return 1;
    }
    else if (pid == 0) { /* child process */
        execlp("/bin/ls", "ls", NULL);
    }
    else { /* parent process */
        /* parent will wait for the child to complete */
        wait(NULL);
        printf("Child Complete");
    }

    return 0;
}
```

Derste gösterilen diğer örnekler:

- **"before fork / after fork":** `fork`'tan önceki `printf` bir kez, sonraki `printf` iki kez yazılır. Çünkü `fork`'tan sonra iki süreç aynı kodu çalıştırır.
- **PID örneği:** İlk süreç PID 2682. `fork` sonrası ebeveyn "PID 2682, fork sonucu 2683", çocuk "PID 2683, fork sonucu 0" yazar. PID'lerin ardışık çıkması tesadüftür.
- **exec örneği:** Çocuk `exec` ile `/bin/ls` programını `/etc/apache2/conf.d` yoluyla çalıştırıyor. "No such file or directory" hatası `exec`'in başarısız olduğu anlamına gelmez. `ls` yüklenmiş ve çalışmıştır, ama o klasör yok ya da erişim izni yok.

Örnekler bilerek basit tutulmuştur. Gerçek bir kabuk için döngü, komut ayrıştırma (parsing) ve parametre ayıklama da gerekir.

exit ve return

- `return` içinde bulunulan fonksiyondan çıkar.
- `exit` süreci, hangi fonksiyonda olursanız olun, sonlandırır. Derin bir fonksiyonda devam edilemeyecek bir hata olursa (ör. `malloc` bellek bulamadıysa) `exit` çağırmak, hatayı her katmanda yukarı taşımaktan daha mantıklıdır.
- `main`'den `return` etmek neredeyse `exit` gibidir, çünkü ardından süreç sonlanır.
- İş parçacığına atanan fonksiyondan `return` etmek o iş parçacığını bitirir (`pthread_exit` gibi). Ama iş parçacığı içinden `exit` çağrılırsa bütün süreç biter.

Bir öğrenci `main`'den `return -1` yapmanın hata üretip üretmediğini sordu. Hoca: hata üretmez, yalnızca sonlandırma durumu döner. Unix'te gelenek, düzgün biten programın 0 döndürmesidir. Sistem çağrıları hata verdiğinde ayrıntı `errno` değişkenine yazılır.

Süreç sonlandırma

Süreç şu yollarla sonlanır:

- Kendi işi biter: `main`'den `return` ya da `exit`.
- Ebeveyn çocuğu sonlandırır. Örnek: kamera → ön işleme → işleme hattını yöneten ana süreç, kullanıcı Ctrl+C'ye basınca çocukları tek tek kapatır.
- Kullanıcı sonlandırır: Ctrl+C ya da pencerenin çarpısına basmak.
- `kill` ile sinyal gönderilir (sinyaller ileride ayrıca işlenecek).

Sonlanan süreç terminated durumuna geçer. Bellek imajı tutulmaz, ama çıkış bilgisi ebeveyn onu toplayana (reap) kadar saklanır.

Düzgün kapanış (graceful shutdown): Süreç kapanırken geçici dosyaları, paylaşımlı bellek alanlarını, semafor kilitlerini vb. temizlemelidir. Böylece kaynakları paylaşan diğer süreçler sorun yaşamaz. Bunun için `atexit` ile sonlanmada çalışacak bir fonksiyon kaydedilir.

Zombi ve öksüz süreçler

Zombi örneği (`zombie`): Ebeveyn `fork`'tan sonra `while (1)` döngüsüne giriyor. Çocuk mesaj yazıp `exit` ediyor. `ps` çıktısında ebeveyn PID 3380, çocuk PID 3381 ve yanında `<defunct>` yazıyor. Çocuk bitmiş ama ebeveyn onu `wait` ile toplamadığı için kalıntısı duruyor.

Öksüz örneği (`zombie2`): Tersi. Ebeveyn hemen bitiyor, çocuk döngüde kalıyor. Çocuğun (PID 3467) ebeveyn PID'si 1 oluyor. İşletim sistemi öksüz çocuğu `init`'e bağlıyor; çocuğun sonucunu `init` toplayacak.

`wait` ile toplama örneği: `NUMBER_OF_CHILD 5` makrosu ile döngüde 5 çocuk yaratılıyor. PID'ler bir dizide tutuluyor. Her çocuk `exit(100 + i)` ile çıkıyor. Ebeveyn `wait(&child_status)` ile her birini topluyor ve çıkış kodlarını (100-104) yazıyor. Çıktıdaki sıra hem `fork` sonrası zamanlama kararına hem de `wait`'in herhangi bir bitmiş çocukla dönmesine bağlıdır.

Çok süreçli mimari: Google Chrome

Chrome her şeyi tek süreçte çalıştırmaz:

- **Tarayıcı (browser) süreci:** kullanıcı arayüzü, disk ve ağ G/Ç'si.
- **Oluşturucu (renderer) süreçleri:** web sayfasını işler (HTML, JavaScript). Kum havuzunda (sandbox) çalışır. Genelde her sekme için ayrıdır.
- **Eklenti (plug-in) süreçleri:** her eklenti türü için ayrı.

Amaç yalıtımdır (isolation). Sorunlu bir sayfa ya da hatalı bir eklenti yalnızca kendi sürecini etkiler.

Süreçler arası iletişim (IPC)

- **Bağımsız süreç:** diğer süreçleri etkilemez, onlardan etkilenmez. Örnek: açılışta başlayan bazı hizmetler.
- **İşbirlikçi süreç:** diğer süreçlerle veri paylaşır, etkileşir. Örnek: kabuk, işi parçalara bölen çok süreçli programlar, işleme hatları (pipeline).

İşbirlikçi süreçlerin haberleşmesi gerekir. İşletim sistemi bunun için iki yöntem sunar:

Paylaşımlı bellek: Süreçler fiziksel olarak paylaşılan bir bellek alanına yazar ve okur.

- Kimin nereye ne yazdığı kontrol edilmelidir.
- Örnek: üretici-tüketici (producer-consumer). Kameradan görüntü alan üretici, görüntüyü işleyen tüketici. Üretici hızlı, tüketici yavaşsa veri kaybolabilir ya da üzerine yazılabilir. Senkronizasyon mekanizmaları gerekir.

Mesaj geçirme: Bilgi mesajlara bölünür ve süreçler arasında aktarılır.

Modern işletim sistemleri ikisini de sunar. Seçim ihtiyaca göre yapılır:

Ölçüt	Mesaj geçirme	Paylaşımlı bellek
Dağıtık sistem	Uygun	Genelde uygun değil
Veri boyutu	Küçük veriler	Büyük veriler
Sistem çağrısı	Her mesajda (kullanıcı/çekirdek geçişi)	Yalnızca alanı oluştururken
Hız	Daha yavaş	Oluşturulduktan sonra hızlı (alan iki sürecin adres uzayına eşlenmiştir)

Ders sonrası notlar

- **`abort`:** Derste "ebeveyn `abort` sistem çağrısıyla çocuğu sonlandırır" denildi. Kitap burada genel bir "abort" işleminden söz eder. POSIX'te `abort()` fonksiyonu çağıran sürecin **kendisini** sonlandırır (SIGABRT). Ebeveyn bir çocuğu sonlandırmak için `kill(pid, SIGTERM)` gibi bir sinyal kullanır.

- **`main`'den `return -1`:** Çıkış durumu 8 bittir. `return -1` kabukta 255 olarak görünür. 0'dan farklı olduğu için gelenek gereği "başarısız" anlamına gelir. Bunu küçük bir programla denedim: `main`'den `return -1` eden programın kabuktaki çıkış kodu 255 çıktı.
- **`fork` sonrası farklı olan ikinci bilgi:** Bu derste sorulan sorunun cevabı ebeveyn süreç kimliğidir (PPID). Çocuğun PPID'si ebeveynin PID'sine eşittir. Ayrıca `fork`'un dönüş değeri de ikisinde farklıdır. Denemede ebeveyn `fork` sonucu olarak çocuğun PID'sini, çocuk 0 aldı; çocuğun `getppid()` değeri ebeveynin PID'siydi.
- **SPARC örneği:** SPARC'ın birden çok yazmaç penceresi (register windows) asıl olarak fonksiyon çağrılarını hızlandırmak için vardır. Kitaptaki örnek, bazı işlemcilerin birden fazla yazmaç seti sağlayarak bağlam değişimini hızlandırabileceğidir (büyük olasılıkla UltraSPARC). "32 kopya" sayısını hoca da "yanlış hatırlamıyorsam" diye verdi; işlemciye göre değişir.
- **iOS sürümü:** Hoca kısıtın "iOS 4 ve öncesi" için geçerli olduğunu, sonra değiştiğini söyledi. Doğrusu: sınırlı arka plan çok görevliliği büyük olasılıkla iOS 4 ile geldi; kısıt iOS 4'ten **önceki** sürümler içindir.
- **Öksüz süreçler:** Bugünkü Linux sistemlerinde öksüz süreç her zaman PID 1'e bağlanmayabilir. `systemd` kullanan sistemlerde bir "subreaper" süreç (ör. kullanıcı oturumunun `systemd` süreci) onu sahiplenebilir. Kitaptaki ve dersteki anlatım klasik UNIX davranışıdır.
- **`atexit`:** Altyazıda "add exit" diye geçiyor. Fonksiyonun adı `atexit`'tir; bir sistem çağrısı değil, C standart kütüphanesi fonksiyonudur.
- **Altyazı hataları:** "inter-hap" = interrupt; "Redicue" = ready queue; "şehit memory" = shared memory; "Gatred Demon" = `kthreadd`.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.